

AWS Data Protection, Encryption & Privacy Compliance

Cloud Security - DEPI

Agenda

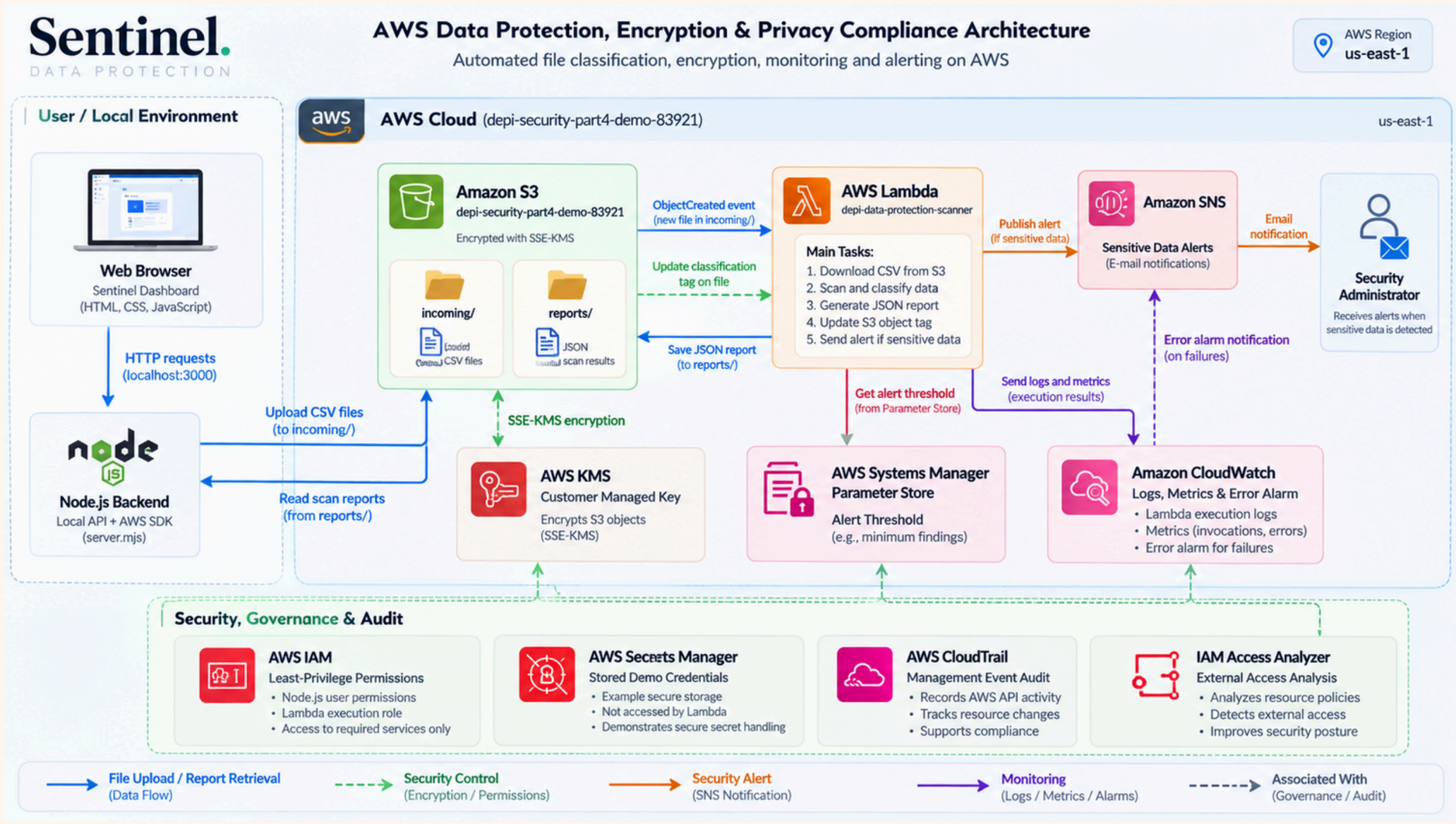
- **Week 1: Data Lifecycle, Classification & S3 Security**
  - **Data Preparation and Classification** (Data Classification Scheme + Data Lifecycle)
  - **Creating and Securing an S3 Bucket** (Block Public Access + Encryption + Versioning)
  - **Configuring Access Policies** (Bucket Policy + Access Point + Access Analyzer)
  - **Data Lifecycle and Testing** (Lifecycle Rules + Access Review + Evidence)

- **Week 2: Encryption at Rest & Key Management**
  - KMS Key Hierarchy & Key Policies
  - Encryption at Rest For targeted services and data stores
  - Key Rotation & Auditing
  - Encryption Testing and Evidence Collection

- **Week 3: Encryption in Transit, Secrets & API Protection**
  - TLS Configuration & AWS Certificate Manager (ACM)
  - Certificate Inventory
  - Secrets Management & Rotation
  - API Security Controls
  - Documentation of Secure Connection Design

- **Week 4: Data in Use, DLP, Privacy & Compliance**
  - Data Discovery & Amazon Macie
  - DLP Plan
  - Privacy & Compliance Mapping
  - Risk Register & Security Review
  - Consolidation of Findings and Final Report

- **Architicture Diagram**



Week 1: Data Lifecycle, Classification & S3 Security

Our goal of this week is classifying data,
mapping cloud data lifecycle: creation, storage, use, sharing, archive, destruction.
Secure S3: bucket policies, versioning, encryption, access points, Access Analyzer, Block Public Access.

1. **Data Preparation and Classification** (Data Classification Scheme + Data Lifecycle)

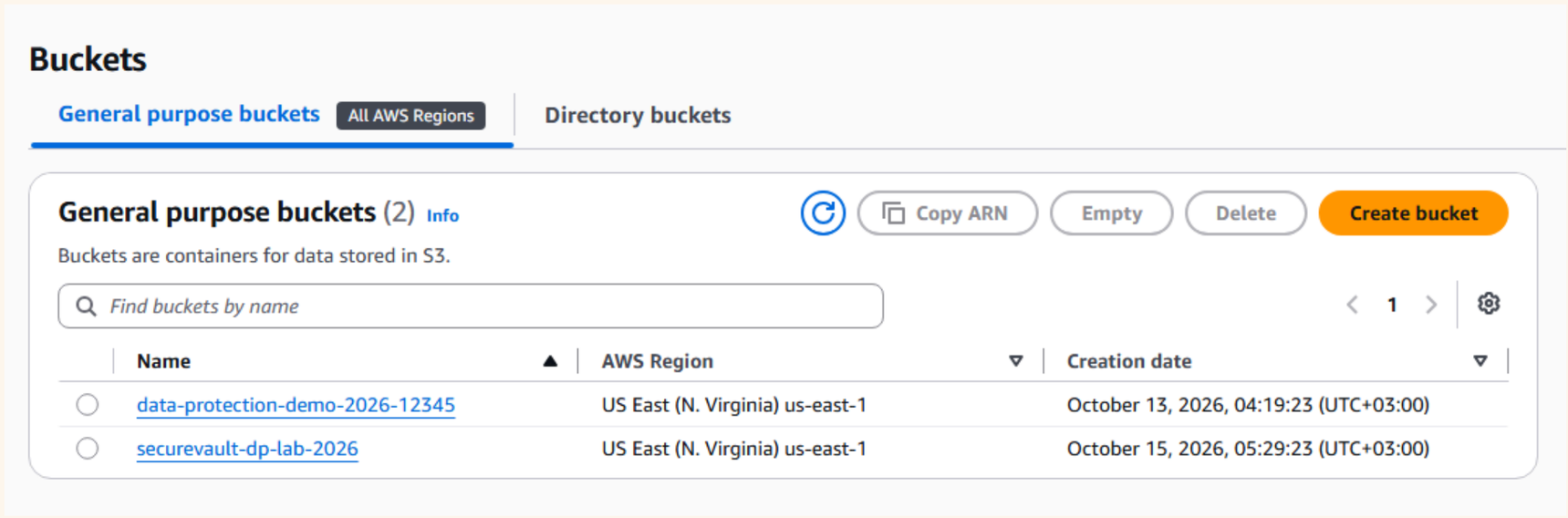
Data classification scheme

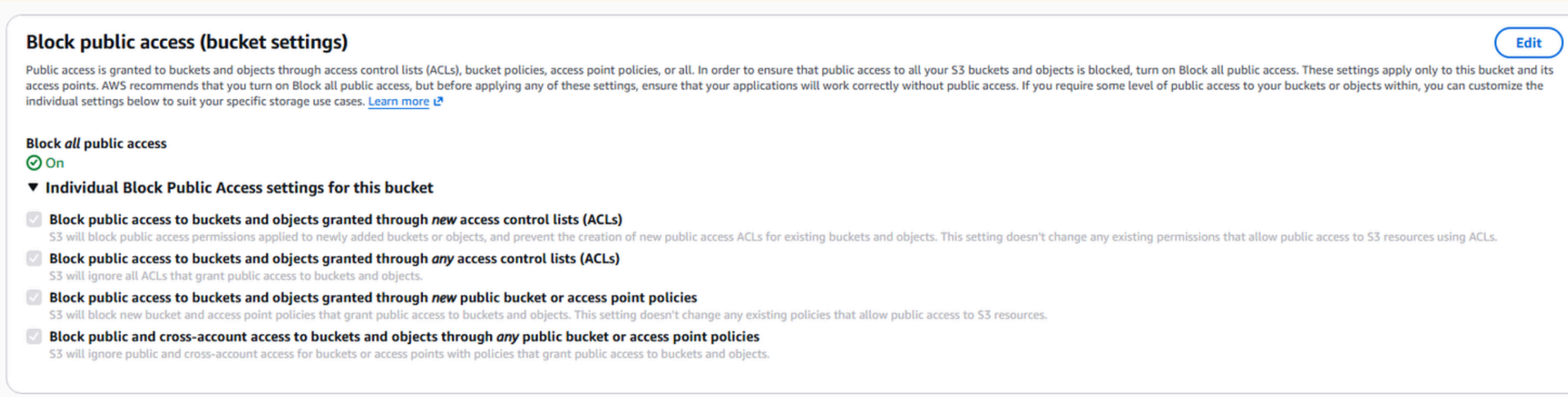
Data Type	Classification	Examples	Access Requirements	Encryption Requirements	Storage	Sharing Requirements	Retention	Disposal
Public Data	**Public**	Website content, public documentation	Public access permitted	Optional	S3	May be shared externally	Based on business need	Standard deletion
Internal Data	**Internal**	Internal documentation, application logs	Employees / authorized users	Required	S3 / CloudWatch	Do not publish publicly	Based on organizational policy	Secure deletion
Customer Data	**Confidential**	Customer name, email, phone	Authorized employees and services only	Required	S3 / RDS	Share only with authorized parties	Defined retention period	Secure deletion
Business Data	**Confidential**	Internal reports, business documents	Authorized employees	Required	S3 / RDS	Share only with authorized parties	Defined retention period	Secure deletion
Credentials	**Restricted**	Passwords, API keys, access tokens	Strictly limited / least privilege	Required	Secrets Manager	Never share through public or insecure channels	Minimum necessary period	Secure deletion / rotation
Encryption Keys	**Restricted**	KMS key-related sensitive configuration	Security / authorized admin roles	AWS KMS controls	AWS KMS	Strictly controlled	Based on key lifecycle	Controlled key deletion
Payment Data	**Restricted**	Card/payment-related information	Strictly limited	Required	Approved encrypted datastore	Strictly controlled	Defined retention period	Secure deletion

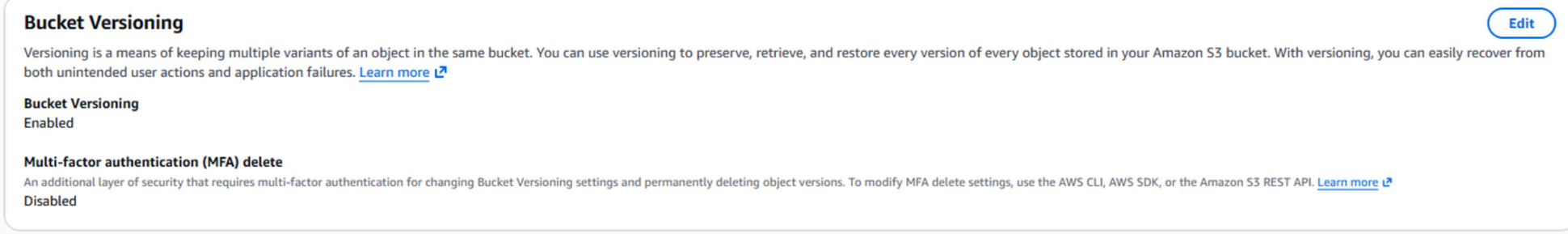
Data Lifecycle Policy

Lifecycle Stage	Policy / Requirement	Key Controls	Public	Internal	Confidential	Restricted
1. Creation	Classify data when created and identify the Data Owner.	Data Classification; Data Owner	Classify before publication	Classify and identify owner	Classify and identify owner	Classify and identify owner; strict handling
2. Storage	Store data in an approved service with controls appropriate to its classification.	S3, Encryption, IAM, Access Control	Approved storage; public access only if explicitly intended	Access-controlled storage	Encrypted storage; restricted access	Highly protected storage; strict access control
3. Use	Allow access only to users/services that require the data for their duties.	IAM. Least Privilege. Bucket Policy	General access as intended	Authorized users	Least privilege	Strict least privilege
4. Sharing	Share data only with authorized parties and through approved secure methods.	Resource Policies, Access Points, Secure Transport, Logging	Allowed if intentionally public	Authorized users/parties	Authorized parties only	Strictly controlled sharing
5. Archive	Move data that is no longer frequently used to an appropriate archival state according to retention requirements.	S3 Lifecycle, Glacier Storage Classes, Retention	Optional	As required	According to retention	According to retention; controlled access
6. Destruction	Delete data securely after the approved retention period and required review.	Deletion Controls, Versioning Review, Audit Evidence	Standard deletion	Secure deletion	Secure deletion	Controlled secure deletion

2. **Creating and Securing an S3 Bucket** (Block Public Access + Encryption + Versioning)

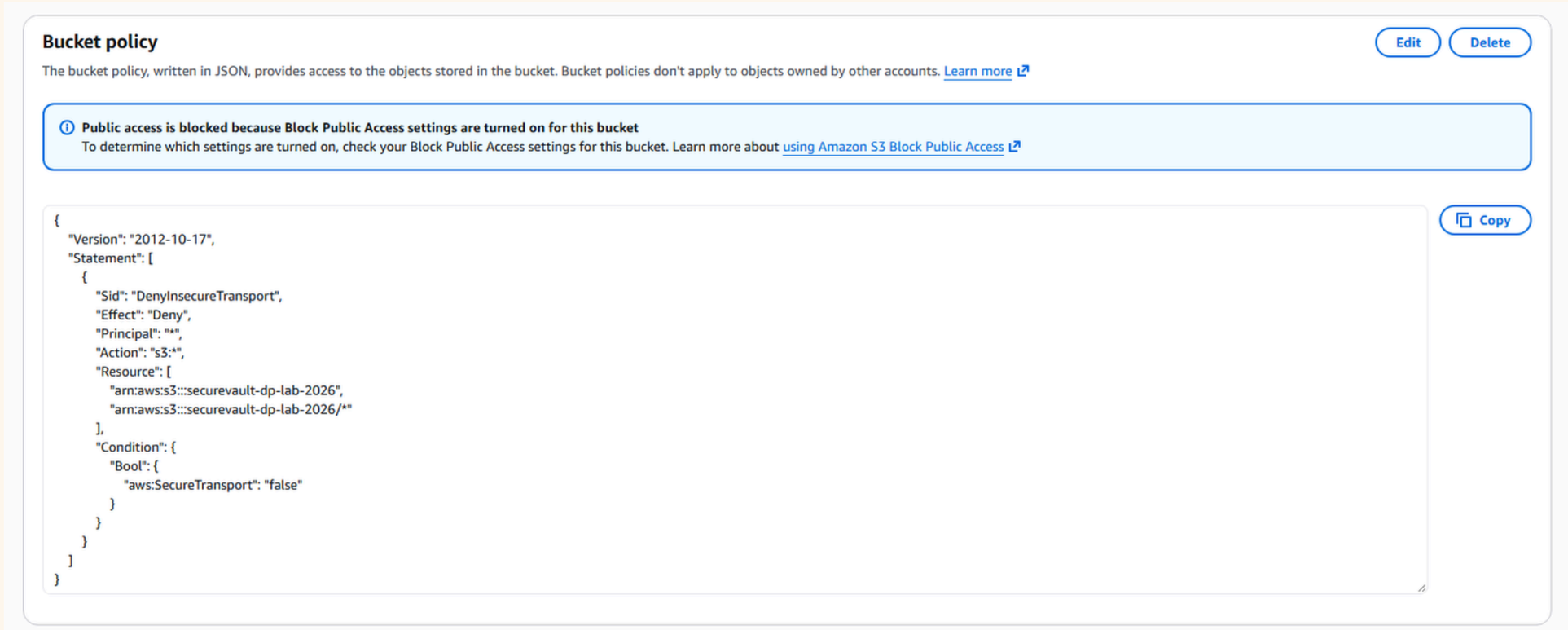




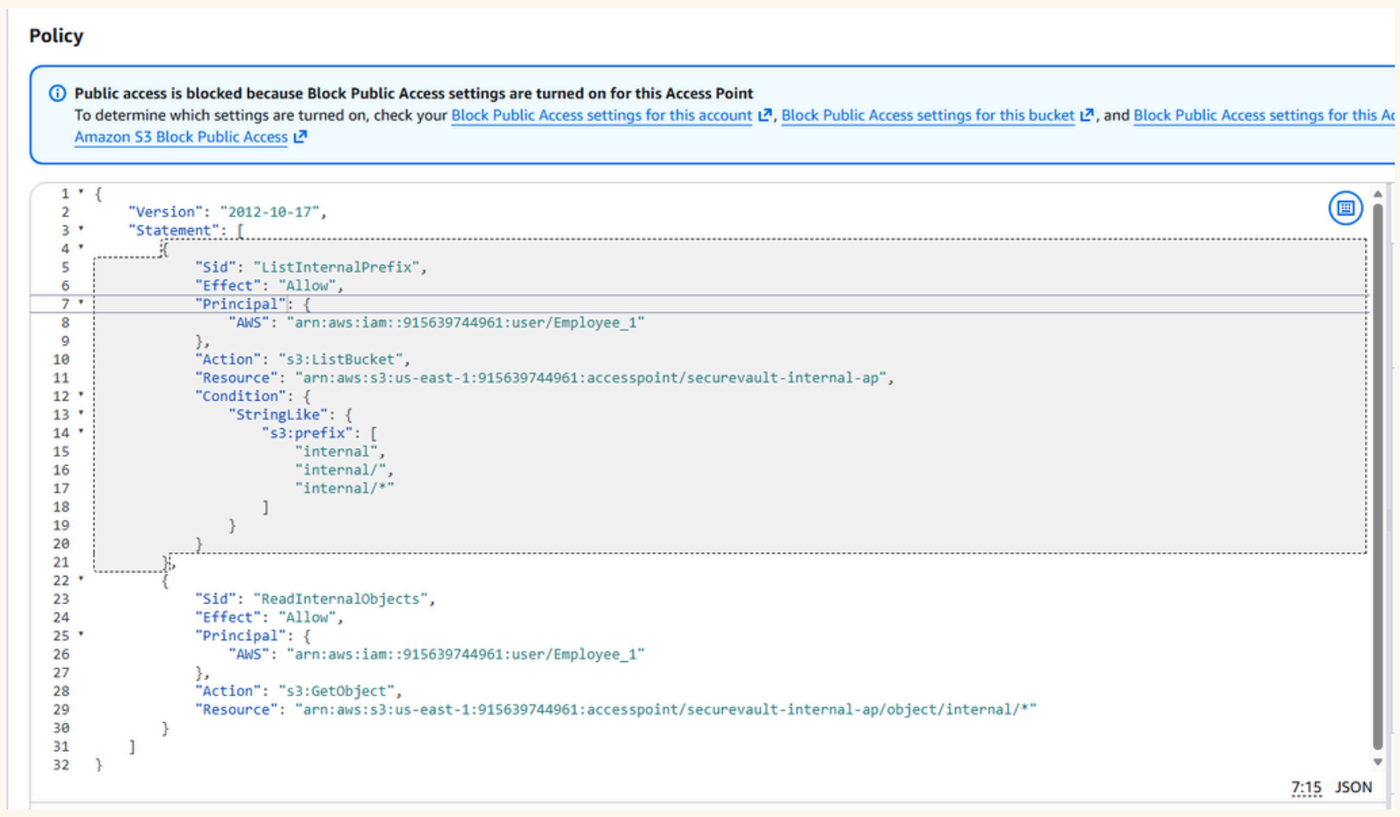


3. **Configuring Access Policies** (Bucket Policy + Access Point + Access Analyzer)

- This Bucket Policy for block any http requests



- If we have an employee who does not have IAM permissions for S3, will adding an access point policy for them grant them access? Nooo




```
PS C:\Users\moham> aws s3api get-object `
>>   --bucket securevault-dp-lab-2026 `
>>   --key internal/internal-procedure.txt `
>>   .\internal-procedure.txt `
>>   --region us-east-1 `
>>   --profile employee_1

aws: [ERROR]: An error occurred (AccessDenied) when calling the GetObject operation: User: arn:aws:iam::915639744961:use
r/Employee_1 is not authorized to perform: s3:GetObject on resource: "arn:aws:s3:::securevault-dp-lab-2026/internal/inte
rnal-procedure.txt" because no identity-based policy allows the s3:GetObject action
```

- What if he has S3 permissions, but no access point policy has been assigned to him? Yes only if has kms:Decrypt permission

```
PS C:\Users\moham> aws s3api get-object `
>>   --bucket securevault-dp-lab-2026 `
>>   --key internal/internal-procedure.txt `
>>   .\internal-procedure.txt `
>>   --region us-east-1 `
>>   --profile employee_2

aws: [ERROR]: An error occurred (AccessDenied) when calling the GetObject operation: User: arn:aws:iam::915639744961:use
r/Employee_2 is not authorized to perform: kms:Decrypt on resource: arn:aws:kms:us-east-1:915639744961:key/ae8ace96-b7de
-4def-9ae1-9b26265feecf because no identity-based policy allows the kms:Decrypt action
```

Before kms:Decrypt permission

```
PS C:\Users\moham> aws s3api get-object `
>>   --bucket securevault-dp-lab-2026 `
>>   --key internal/internal-procedure.txt `
>>   .\internal-procedure.txt `
>>   --region us-east-1 `
>>   --profile employee_2
{
    "AcceptRanges": "bytes",
    "LastModified": "2026-10-15T03:06:01+00:00",
    "ContentLength": 117,
    "ETag": "\"c2d131dc237759f329d02b52eab5574d\"",
    "ChecksumCRC64NVME": "TYknjcUHfpY=",
    "ChecksumType": "FULL_OBJECT",
    "VersionId": "_uSSWak27wV1L7jZPygVtTxGzXxQ149W",
    "ContentType": "text/plain",
    "ServerSideEncryption": "aws:kms",
    "Metadata": {},
    "SSEKMSKeyId": "arn:aws:kms:us-east-1:915639744961:key/ae8ace96-b7de-4def-9ae1-9b26265feecf",
    "BucketKeyEnabled": true
}
```

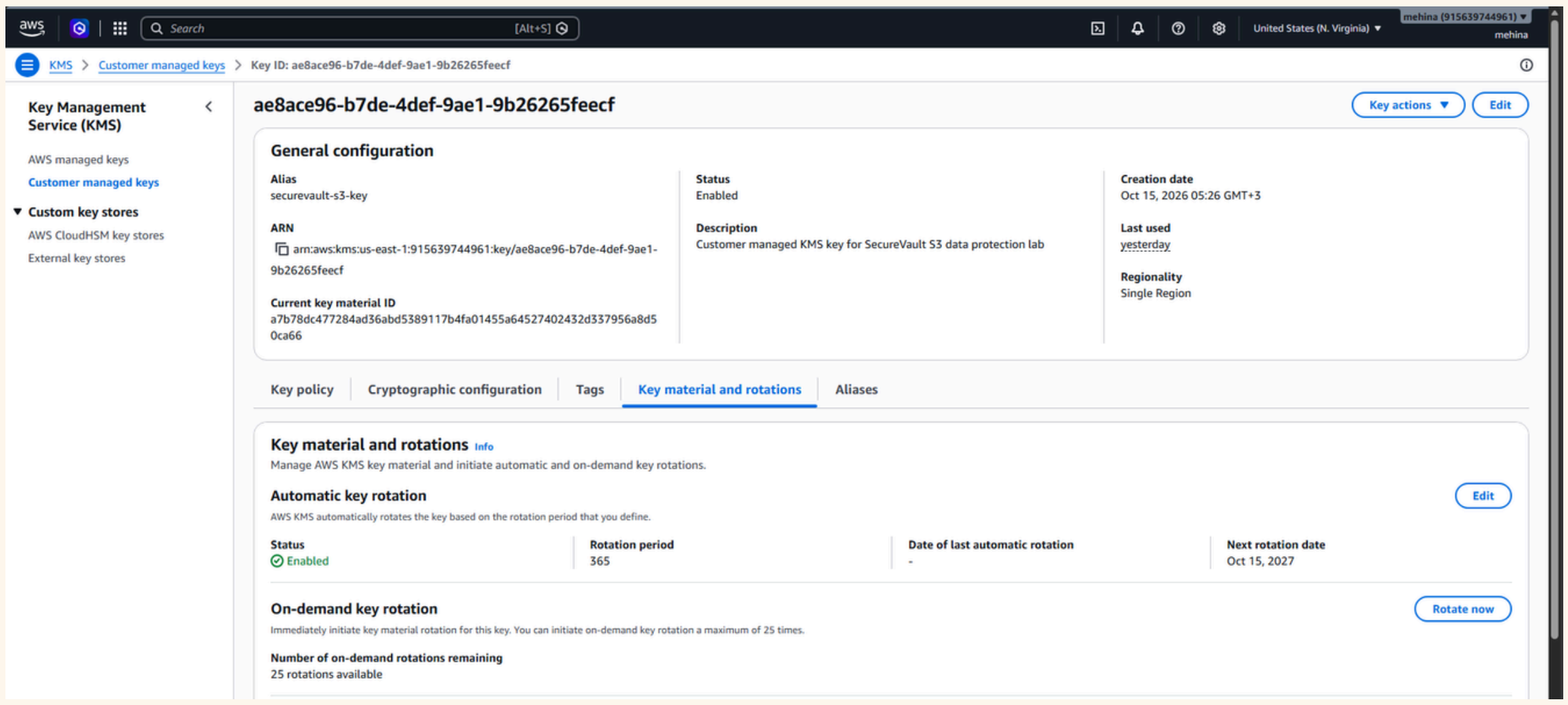
After permission

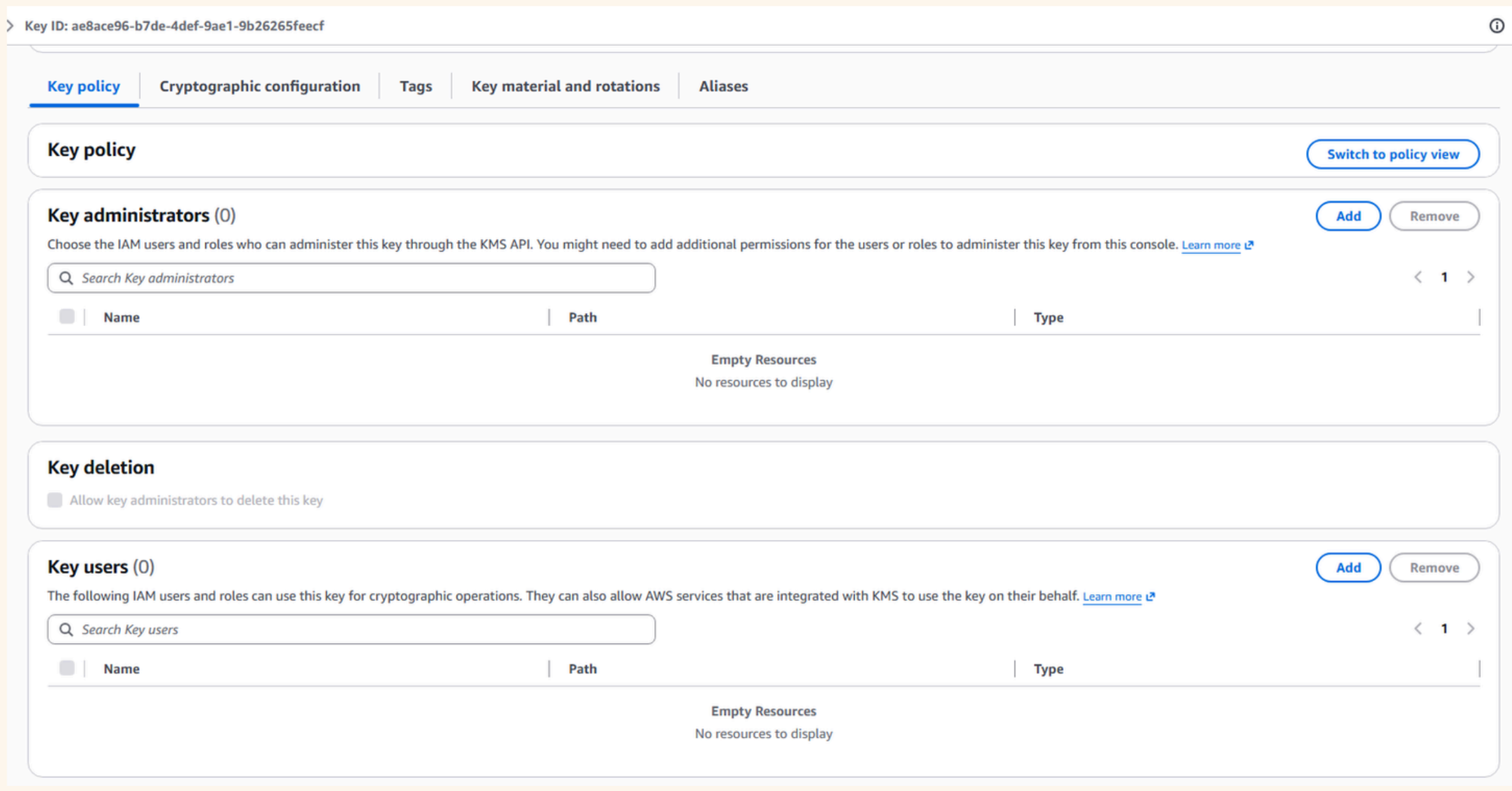
Week 2: Encryption at Rest & Key Management

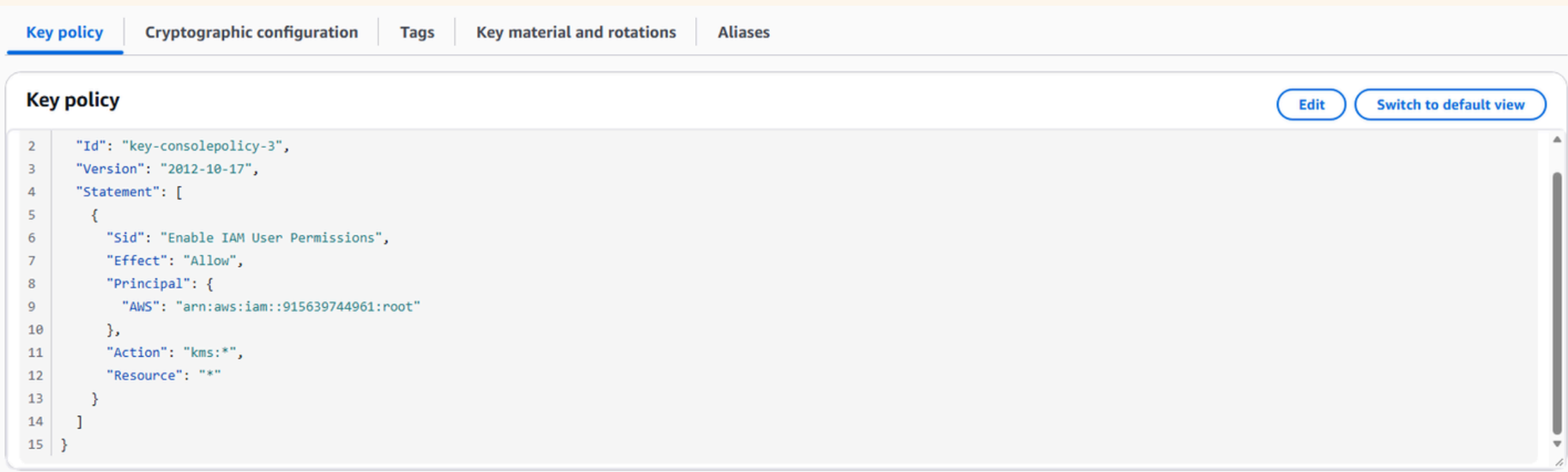
Our goal this week is to implement encryption for data at rest and manage encryption keys.

Customer Managed keys

- securevault-s3-key







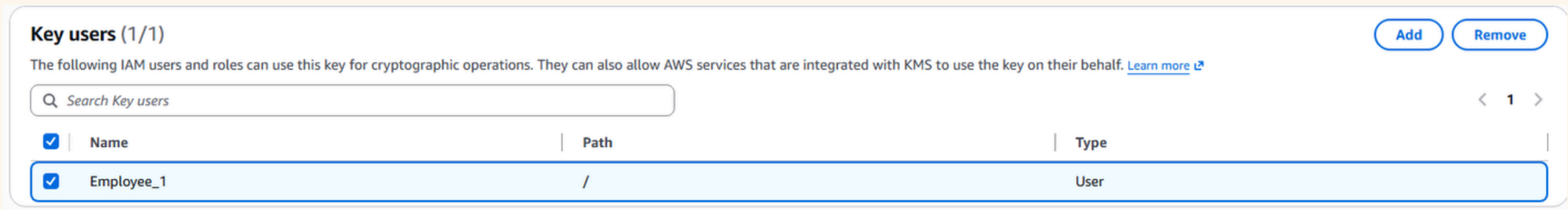
Before Key USER & Key ADMINISTRATOR

```
PS C:\Users\moham> aws kms encrypt `
>>   --key-id arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238 `
>>   --plaintext fileb://plain.txt `
>>   --region us-east-1 `
>>   --profile employee_1

aws: [ERROR]: An error occurred (AccessDeniedException) when calling the Encrypt operation: User: arn:aws:iam::915639744961:user/Employ
ee_1 is not authorized to perform: kms:Encrypt on resource: arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238
 because no identity-based policy allows the kms:Encrypt action
PS C:\Users\moham>
PS C:\Users\moham>
PS C:\Users\moham> aws kms disable-key `
>>   --key-id arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238 `
>>   --region us-east-1 `
>>   --profile employee_1

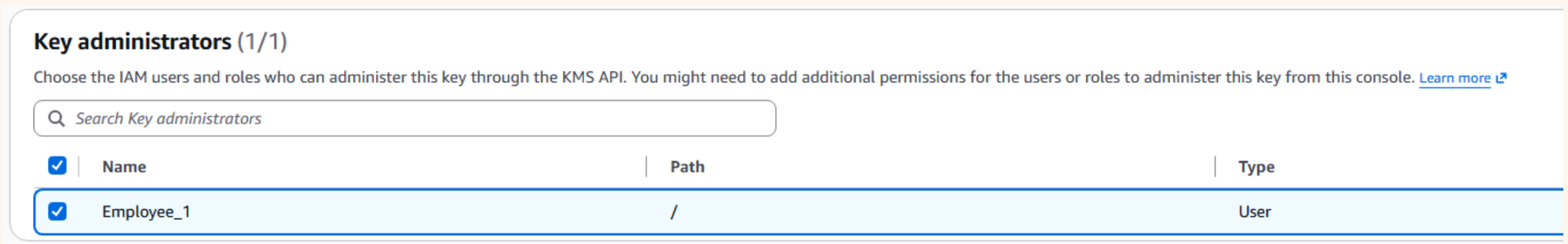
aws: [ERROR]: An error occurred (AccessDeniedException) when calling the DisableKey operation: User: arn:aws:iam::915639744961:user/Emp
loyee_1 is not authorized to perform: kms:DisableKey on resource: arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a
0c3238 because no identity-based policy allows the kms:DisableKey action
PS C:\Users\moham>
PS C:\Users\moham>
```

After Key USER




```
PS C:\Users\moham> aws kms encrypt `
>>   --key-id arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238 `
>>   --plaintext fileb://plain.txt `
>>   --region us-east-1 `
>>   --profile employee_1
{
    "CiphertextBlob": "AQICAHjeYjext8ftYrXB1YlwsT3zFrHHycvkclX0qnWBdkINNgGgwx8POWSaeJk71Ppns688AAAAgTB/BgkqhkiG9w0BBwagcjBwAgEAMGsGCSqG
SIb3DQEHATAeBglghkgBZQMEAS4wEQQMauwMAEnwyXVOBdSoAgEQgD4wkr6VNgs0wmEb7ODjrJfngaLnzsmBSV7BUC2rRUs98Vohh2s+9CwRNfjOxY2svClK0v8beb9v6btfFdM
Eug==",
    "KeyId": "arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238",
    "EncryptionAlgorithm": "SYMMETRIC_DEFAULT"
}
```

After Key ADMINISTRATOR




```
PS C:\Users\moham>
PS C:\Users\moham> aws kms disable-key `
>>   --key-id arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238 `
>>   --region us-east-1 `
>>   --profile employee_1
PS C:\Users\moham>
PS C:\Users\moham>
```

☐	testKey	f70fca76-e189-4c57-baa6-072...	Disabled

Removed from USER & ADMINISTRATOR but Added IAM policy to this account

Specify permissions Info

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor Visual

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "KMSLabUsage",
      "Effect": "Allow",
      "Action": [
        "kms:Encrypt",
        "kms:Decrypt",
        "kms:DescribeKey",
        "kms:GetKeyPolicy"
      ],
      "Resource": "arn:aws:kms:us-east-1:915639744961:key/ae8ace96-b7de-4def-9ae1-9b26265feecf"
    }
  ]
}
```

```
PS C:\Users\moham> aws kms get-key-policy `
>>   --key-id arn:aws:kms:us-east-1:915639744961:key/ae8ace96-b7de-4def-9ae1-9b26265feecf `
>>   --policy-name default `
>>   --region us-east-1 `
>>   --profile employee_1
{
    "Policy": "{\n  \"Version\" : \"2012-10-17\",\n  \"Id\" : \"key-consolepolicy-3\",\n  \"Statement\" : [ {\n    \"Sid\" : \"Enable I
AM User Permissions\",\n    \"Effect\" : \"Allow\",\n    \"Principal\" : {\n      \"AWS\" : \"arn:aws:iam::915639744961:root\"\n    },\
n    \"Action\" : \"kms:*\",\n    \"Resource\" : \"*\"\n  } ]\n}",
    "PolicyName": "default"
}

PS C:\Users\moham>
PS C:\Users\moham> aws kms encrypt `
>>   --key-id arn:aws:kms:us-east-1:915639744961:key/ae8ace96-b7de-4def-9ae1-9b26265feecf `
>>   --plaintext fileb://plain.txt `
>>   --region us-east-1 `
>>   --profile employee_1 `
>>   --query CiphertextBlob `
>>   --output text
AQICAHint43EdyhK02q9U4kRe0+gFFWmRSdAJDLTN5VqjVDKZgHtCg0xDyA/he1DRYXfXPI0AAAAgTB/BgkqhkiG9w0BBwagcjBwAgEAMGsGCSqGSIb3DQEHATAeBglghkgBZQM
EAS4wEQQMLq4Frnq0OqIK5EDSAgEQgD6geCgbuYMqUbcF3sSmQonk+3VdiiZuJyhEKhQyDC4D43kBsH6VlEnsorwHkvsygyP9DOqwDCE815sHyY2GoQ==
```

Adding a key policy conflicts with an IAM policy.

Edit key policy

```
{
    "Version": "2012-10-17",
    "Id": "key-consolepolicy-3",
    "Statement": [
        {
            "Sid": "Enable IAM User Permissions",
            "Effect": "Allow",
            "Principal": {
                "AWS": "arn:aws:iam::915639744961:root"
            },
            "Action": "kms:*",
            "Resource": "*"
        },
        {
            "Sid": "ExplicitDenyEmployee1KMSOperations",
            "Effect": "Deny",
            "Principal": {
                "AWS": "arn:aws:iam::915639744961:user/Employee_1"
            },
            "Action": [
                "kms:Encrypt",
                "kms:Decrypt",
                "kms:DescribeKey",
                "kms:GetKeyPolicy"
            ],
            "Resource": "*"
        }
    ]
}
```

```
PS C:\Users\moham> aws kms get-key-policy `
>>   --key-id arn:aws:kms:us-east-1:915639744961:key/ae8ace96-b7de-4def-9ae1-9b26265feecf `
>>   --policy-name default `
>>   --region us-east-1 `
>>   --profile employee_1

aws: [ERROR]: An error occurred (AccessDeniedException) when calling the GetKeyPolicy operation: User: arn:aws:iam::915639744961:user/E
mployee_1 is not authorized to perform: kms:GetKeyPolicy on resource: arn:aws:kms:us-east-1:915639744961:key/ae8ace96-b7de-4def-9ae1-9b
26265feecf with an explicit deny in a resource-based policy
PS C:\Users\moham>
PS C:\Users\moham> aws kms encrypt `
>>   --key-id arn:aws:kms:us-east-1:915639744961:key/ae8ace96-b7de-4def-9ae1-9b26265feecf `
>>   --plaintext fileb://plain.txt `
>>   --region us-east-1 `
>>   --profile employee_1 `
>>   --query CiphertextBlob `
>>   --output text

aws: [ERROR]: An error occurred (AccessDeniedException) when calling the Encrypt operation: User: arn:aws:iam::915639744961:user/Employ
ee_1 is not authorized to perform: kms:Encrypt on resource: arn:aws:kms:us-east-1:915639744961:key/ae8ace96-b7de-4def-9ae1-9b26265feecf
 with an explicit deny in a resource-based policy
PS C:\Users\moham>
```

- **Grant Access**

  A grant is a policy instrument that allows AWS principals or AWS service principals to use KMS keys in cryptographic operations. It can also let them view a KMS key (DescribeKey) and create and manage grants. When authorizing access to a KMS key, grants are considered alongside key policies and IAM policies.

  Grants are often used for temporary permissions — you can create one, use its permissions, and delete it without changing your key policies or IAM policies.

Create Grant for Employee 1 to can use ‘testkey’ for encryption:

```
PS C:\Users\moham>
PS C:\Users\moham> aws kms create-grant `
>> --key-id arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238 `
>> --grantee-principal arn:aws:iam::915639744961:user/Employee_1 `
>> --operations Encrypt `
>> --retiring-principal arn:aws:iam::915639744961:user/admin `
>> --name employee1-encrypt `
>> --region us-east-1 `
>> --profile admin
{
    "GrantToken": "AQpANzM0NzAzMTM3MWU1Zjg5N2I3MDcxZjBhOWU2NGI1ZDlmOWU1NTMwZTdmMGI0ZTg0ZmIxYTg4YmIxZTU2MTU4NyKzAgEBAgB4c0cDE3Hl-Je3Bx8K
nmS12fnlUw5_C06E-xqIux5WFYcAAAEKMIIBBgYJKoZIhvcNAQcGoIH4MIH1AgEAMIHvBgkqhkiG9w0BBwEwHgYJYIZIAWUDBAEuMBEEDFrutojflL6-FXE9tAIBEICBweW3XeQ
POFlW7IZ0lbnARZqh_P2FacpvdGeAtC6OsHrdZ_Kgnk6jIbdpjJGC8igxbMdzQoB5utQFrOhW8oKMKMi_q3BgvpdzZ0tD2q_z5ZxJzAUgiWWGvPsnyqc27bbO-hT2jECl6sGU1T
P3d5pcf5QkJeZt5ZYlReHrUXXBlvMkqTdlaBOEZ_-bw13eWXC8k1kcpL_TB6nR-dgs-ToulKawSCnATOH2HL0NdL87yuuoaAhaC1sHrnirDArf5bFsjjIqIFHHl04XYBa71KBS6
0W7p5hh5gst0qyRZ75RDJNZJcwc",
    "GrantId": "51c7974e176016bbd4a052eb45bba79861e60b2dd2ac9167be510c935925cc1c"
}

PS C:\Users\moham>
PS C:\Users\moham>
PS C:\Users\moham> aws kms list-grants `
>>   --key-id arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238 `
>>   --region us-east-1 `
>>   --profile admin
{
    "Grants": [
        {
            "KeyId": "arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238",
            "GrantId": "51c7974e176016bbd4a052eb45bba79861e60b2dd2ac9167be510c935925cc1c",
            "Name": "employee1-encrypt",
            "CreationDate": "2026-10-17T00:14:46+03:00",
            "GranteePrincipal": "arn:aws:iam::915639744961:user/Employee_1",
            "RetiringPrincipal": "arn:aws:iam::915639744961:user/admin",
            "IssuingAccount": "arn:aws:iam::915639744961:root",
            "Operations": [
                "Encrypt"
            ]
        }
    ]
}

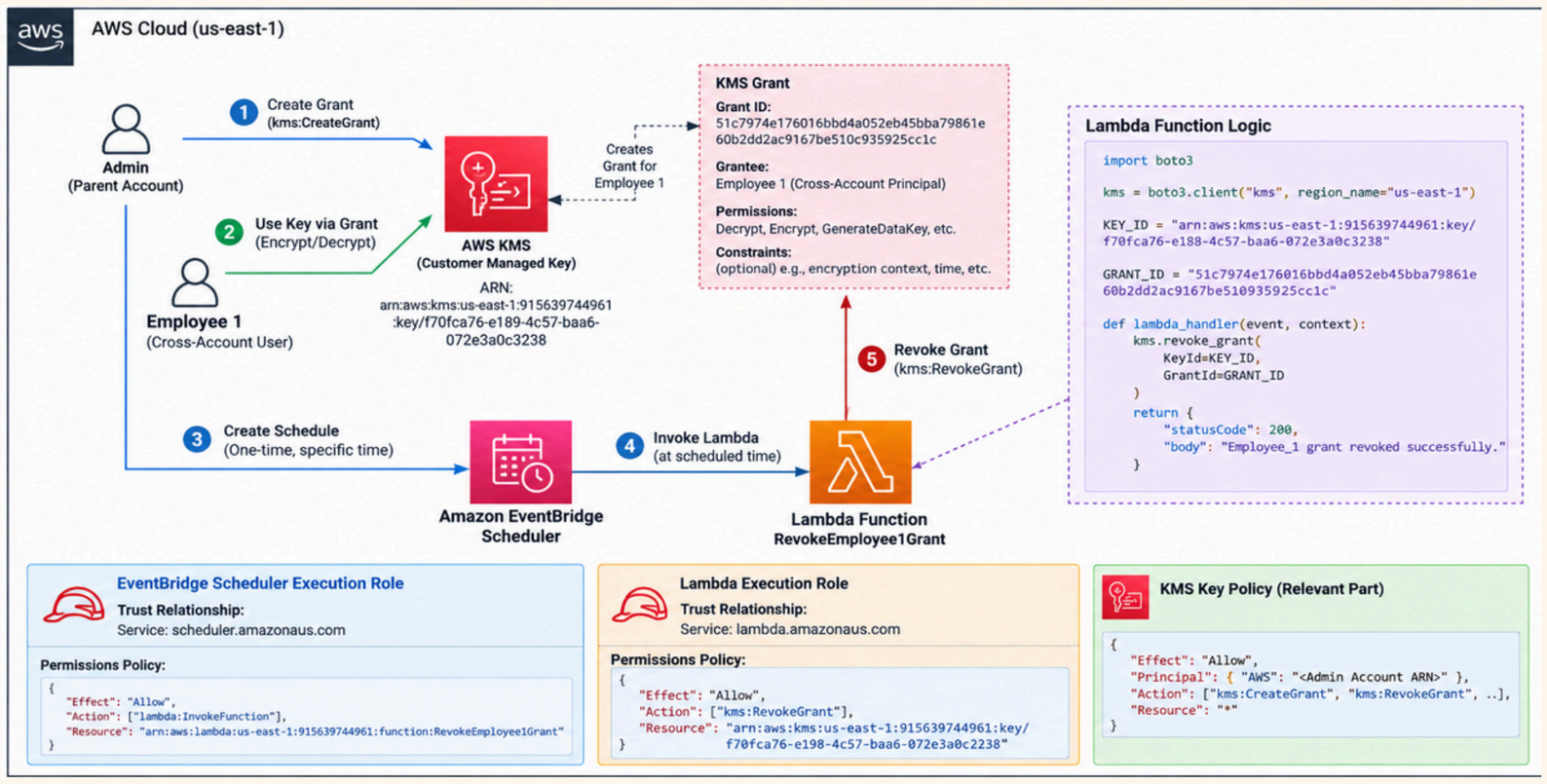
PS C:\Users\moham>
```

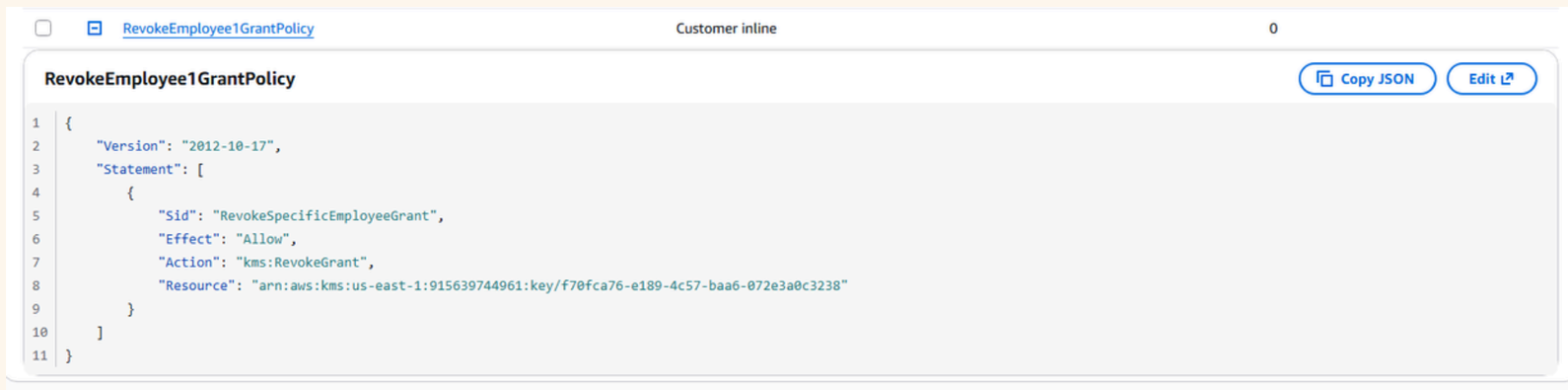
Test:

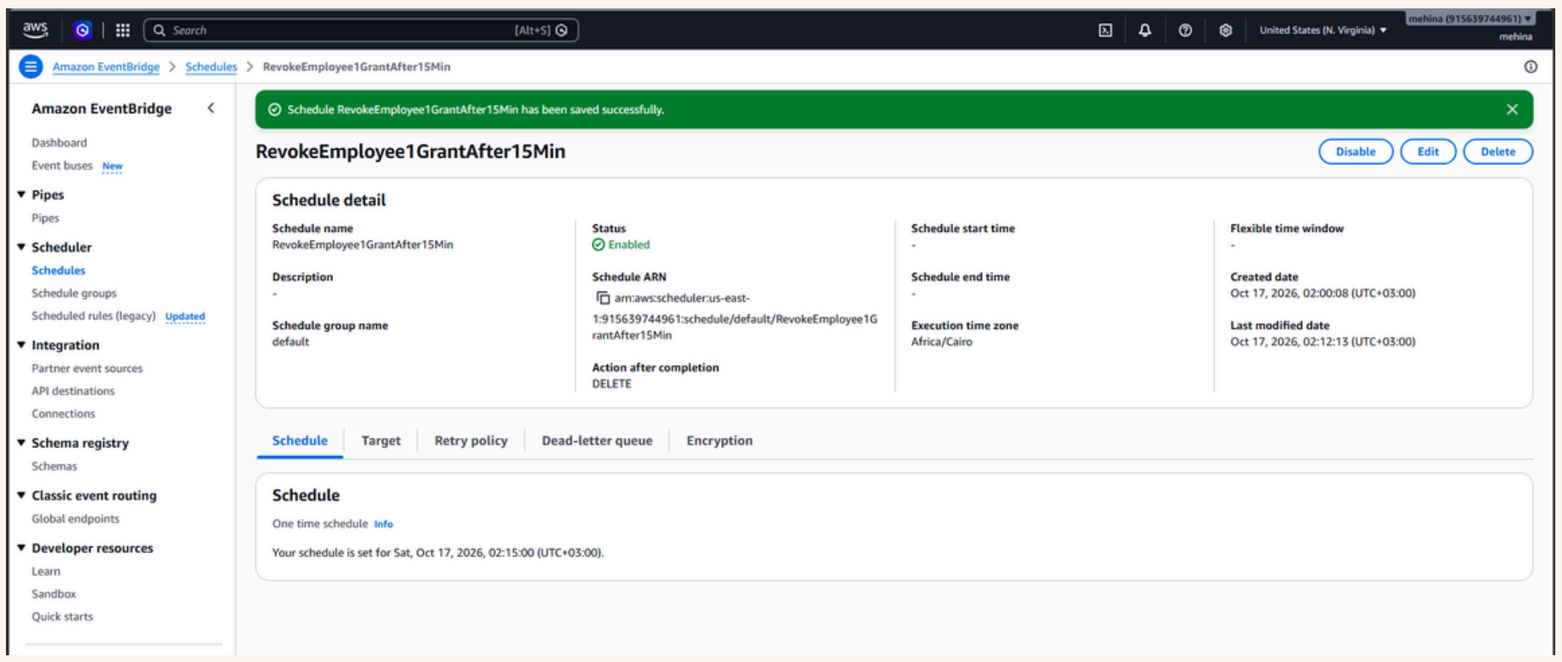
```
PS C:\Users\moham>
PS C:\Users\moham> aws kms encrypt `
>>   --key-id arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238 `
>>   --plaintext fileb://plain.txt `
>>   --grant-tokens AQpANzM0NzAzMTM3MWU1Zjg5N2I3MDcxZjBhOWU2NGI1ZDlmOWU1NTMwZTdmMGI0ZTg0ZmIxYTg4YmIxZTU2MTU4NyKzAgEBAgB4c0cDE3Hl-Je3Bx8
KnmS12fnlUw5_C06E-xqIux5WFYcAAAEKMIIBBgYJKoZIhvcNAQcGoIH4MIH1AgEAMIHvBgkqhkiG9w0BBwEwHgYJYIZIAWUDBAEuMBEEDFrutojflL6-FXE9tAIBEICBweW3Xe
QPOFlW7IZ0lbnARZqh_P2FacpvdGeAtC6OsHrdZ_Kgnk6jIbdpjJGC8igxbMdzQoB5utQFrOhW8oKMKMi_q3BgvpdzZ0tD2q_z5ZxJzAUgiWWGvPsnyqc27bbO-hT2jECl6sGU1
TP3d5pcf5QkJeZt5ZYlReHrUXXBlvMkqTdlaBOEZ_-bw13eWXC8k1kcpL_TB6nR-dgs-ToulKawSCnATOH2HL0NdL87yuuoaAhaC1sHrnirDArf5bFsjjIqIFHHl04XYBa71KBS
60W7p5hh5gst0qyRZ75RDJNZJcwc `
>> --region us-east-1 `
>>   --profile employee_1
{
    "CiphertextBlob": "AQICAHjeYjext8ftYrXB1YlwsT3zFrHHycvkclX0qnWBdkINNgEZ1QIGiwfoWqyRvZD7afLqAAAAgTB/BgkqhkiG9w0BBwagcjBwAgEAMGsGCSqG
SIb3DQEHATAeBglghkgBZQMEAS4wEQQM/LvUNMhe6skSd2yuAgEQgD7UT7UD7lWzAqulXPKD5CkP+Jcc09HZpkSaryjzEwOoQwmss67oHQqAVS79ufQ2C4z1nrJvELcNrQBMLEQ
+kQ==",
    "KeyId": "arn:aws:kms:us-east-1:915639744961:key/f70fca76-e189-4c57-baa6-072e3a0c3238",
    "EncryptionAlgorithm": "SYMMETRIC_DEFAULT"
}

PS C:\Users\moham>
```

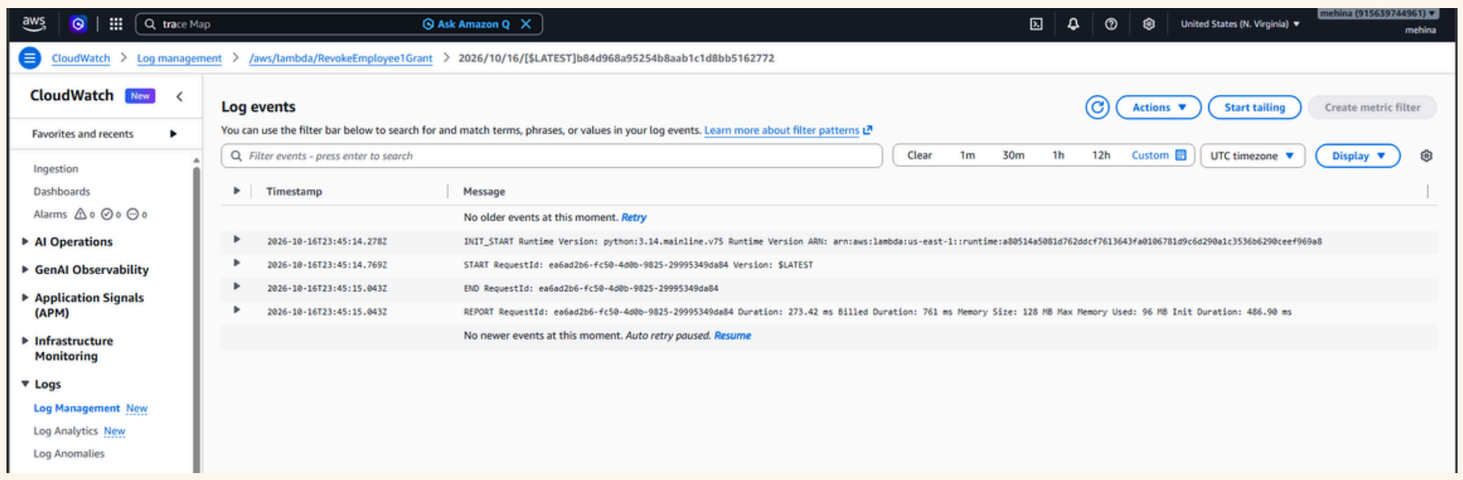
If we want to set an expiration time for this grant token, we can do so by creating a Lambda function and connecting it to AWS EventBridge.







AWS EventBridge



On Time



On Time

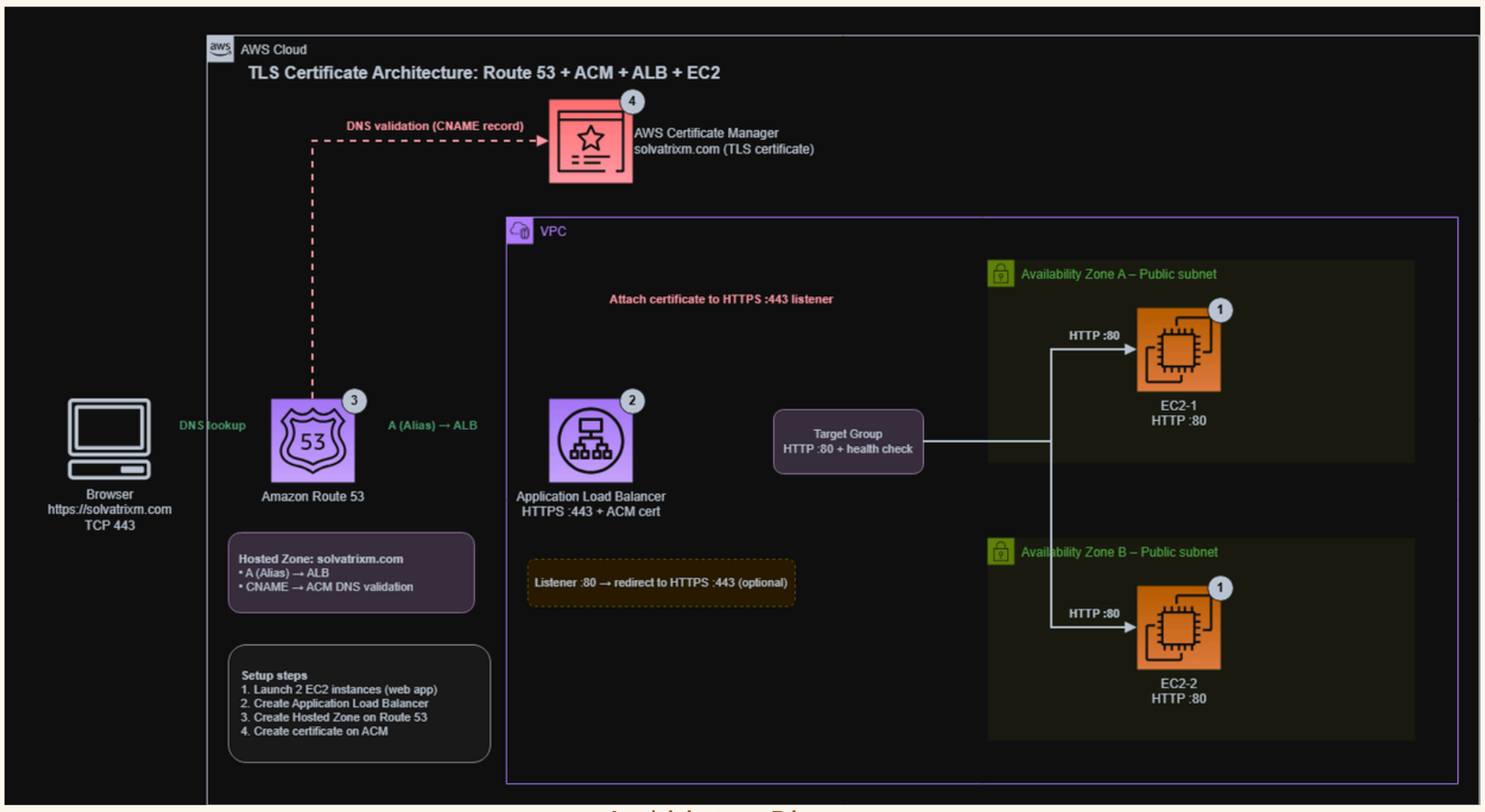
Week 3: Encryption in Transit, Secrets & API Protection

- **TLS Configuration & AWS Certificate Manager (ACM):**

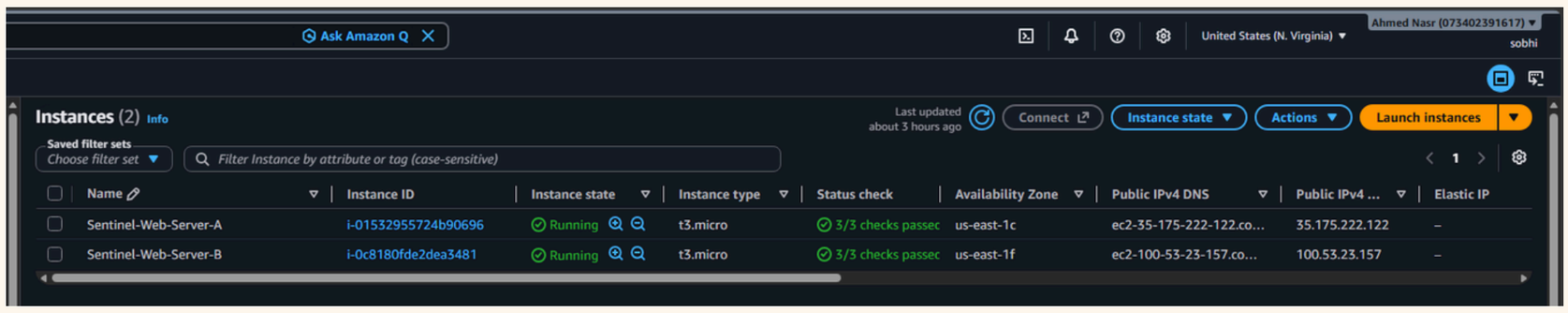
  To run the website over HTTPS ensuring secure data transmission between the site and its users, we must obtain a domain name in order to activate a TLS/SSL certificate.

  STEPS:

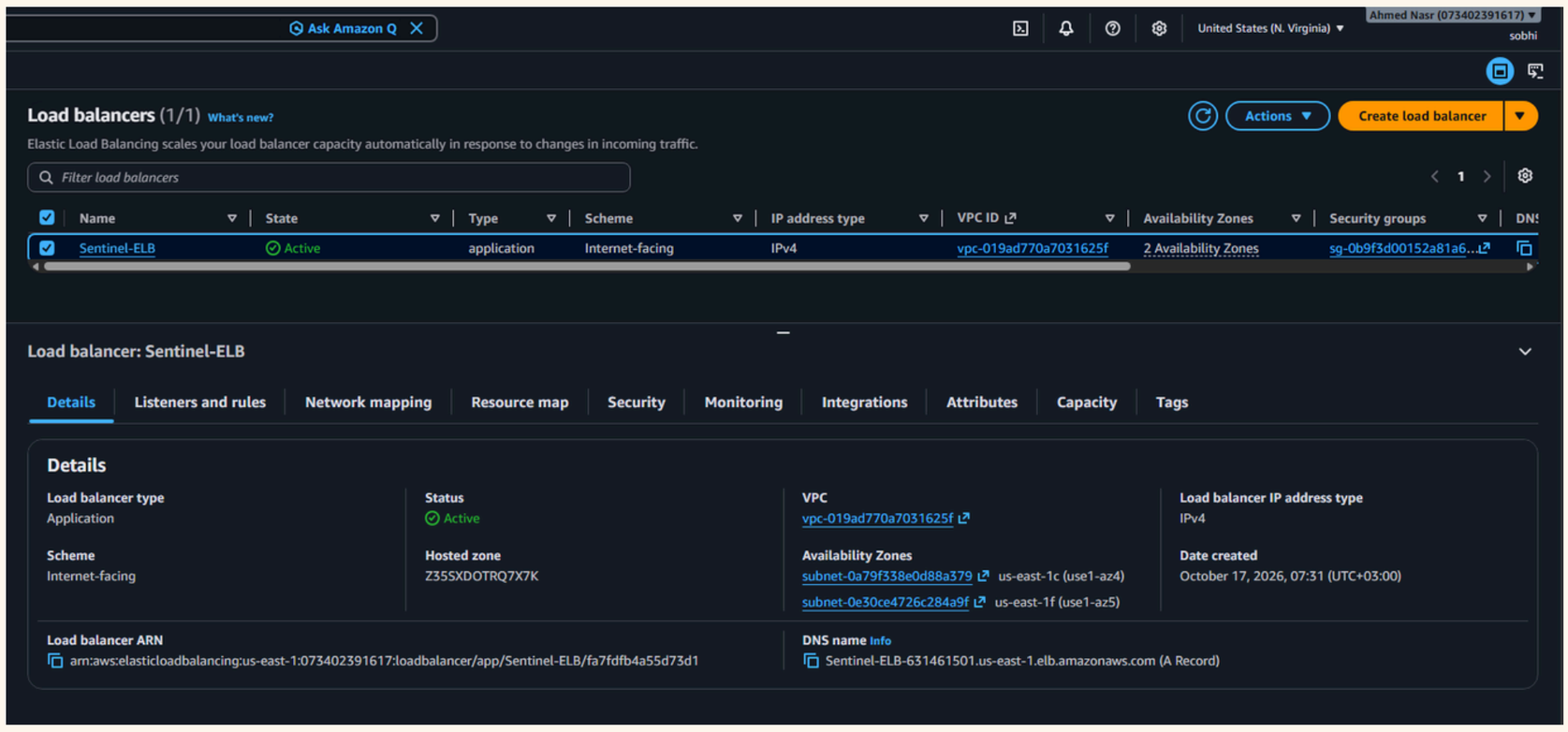
  - Launch 2 **EC2 instances** for our web application.
  - Create **Load Balancer**
  - Create Hosted Zone on **Route53** service
  - Create Certificates on **ACM(AWS Certificate Manager)**



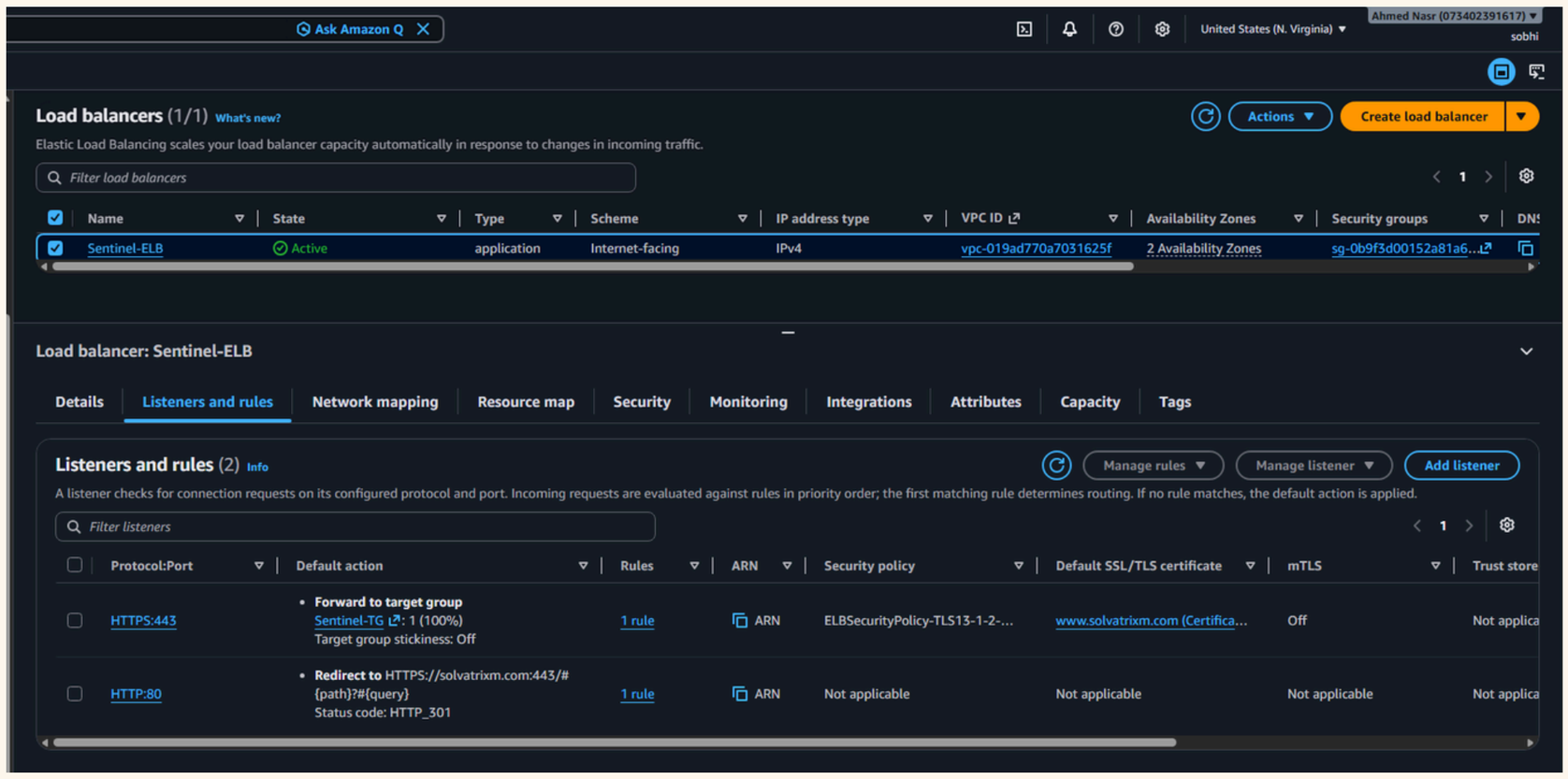
Architicture Diagram



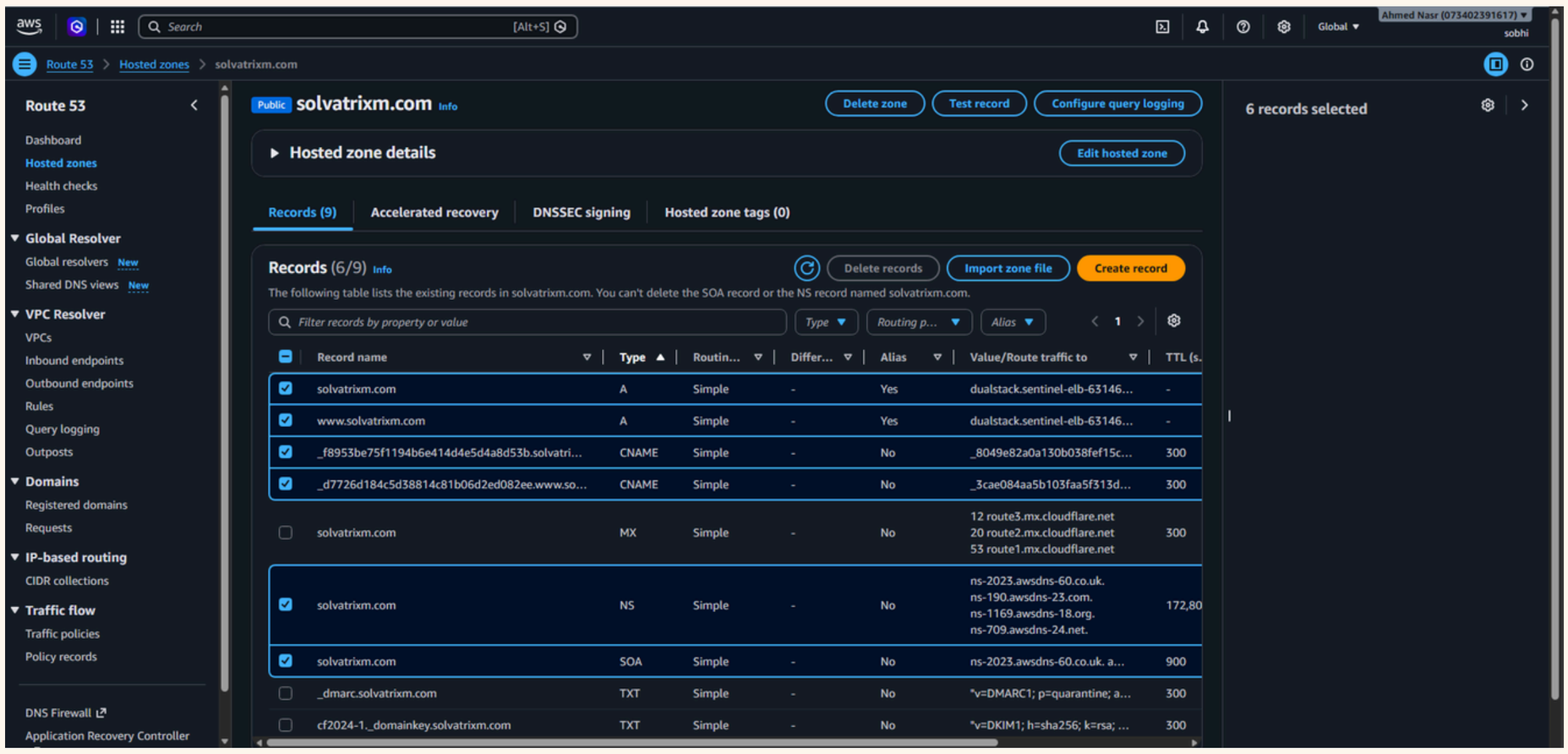
2 instances to ensure hight availability (can do with only one)



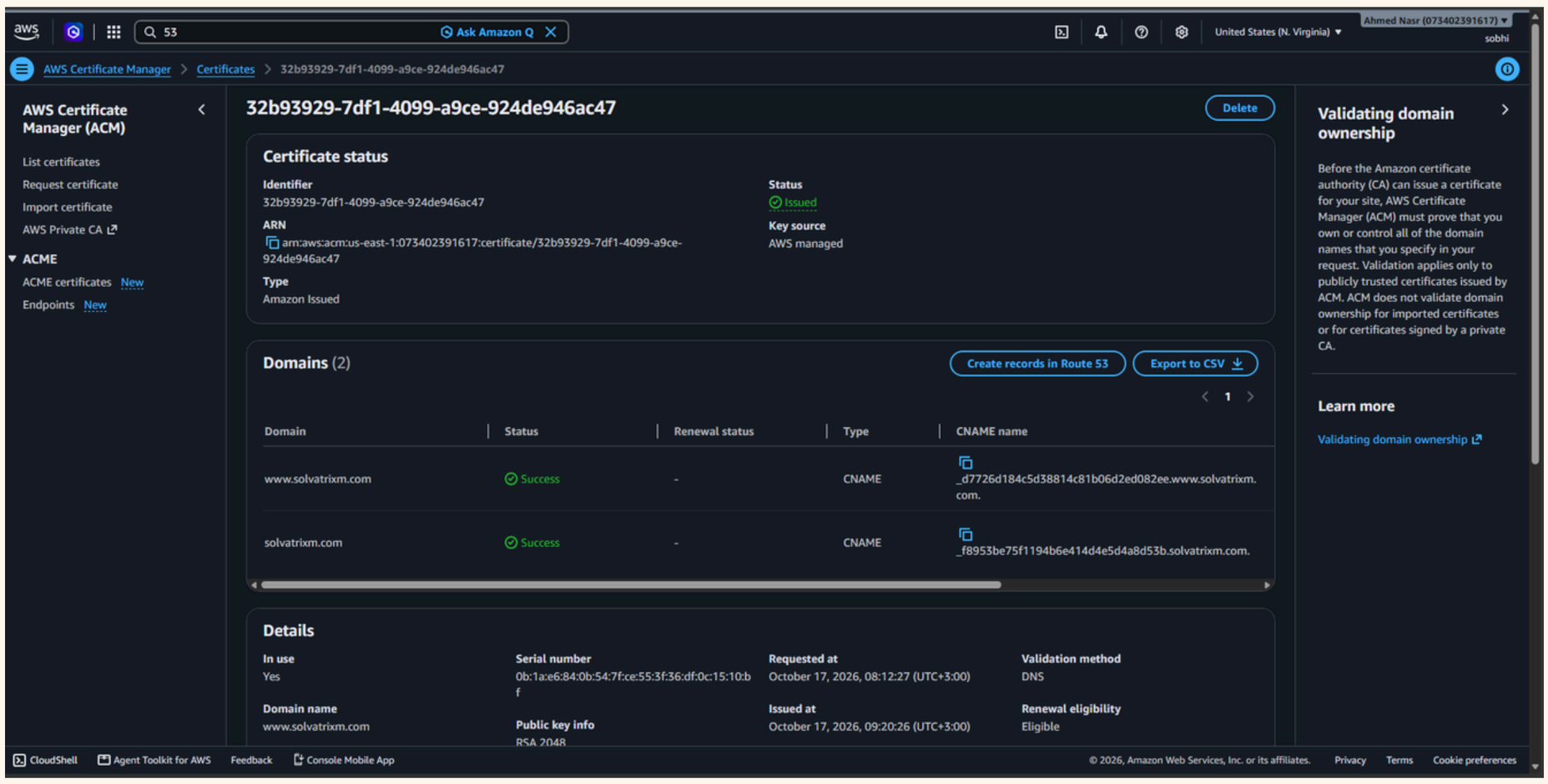
created load balancer and assign 2 instances to it



Handle listener for http/https



Connecting a domain to Route 53 to manage it via AWS



The TLS/SSL certificate has been successfully created. ✅

- **Certificate Inventory**

Field	Details
Certificate Service	AWS Certificate Manager (ACM)
Domain Name	www.solvatrixm.com
Certificate Status	Issued
In Use	Yes
Validation Method	DNS
Public Key Algorithm	RSA 2048
Signature Algorithm	SHA-256 with RSA
Requested At	October 17, 2026, 08:12:27 (UTC+3)
Issued At	October 17, 2026, 09:20:26 (UTC+3)
Valid From	October 17, 2026, 03:00:00 (UTC+3)
Expiration Date	May 3, 2027, 02:59:59 (UTC+3)
Renewal Eligibility	Eligible
Export Option	Disabled
Additional Names	1
Associated Resource	Application Load Balancer (ELB)
AWS Region	us-east-1
Load Balancer Name	Sentinel-ELB
Certificate ARN	Not visible in the screenshot

- **Secrets Management & Rotation**

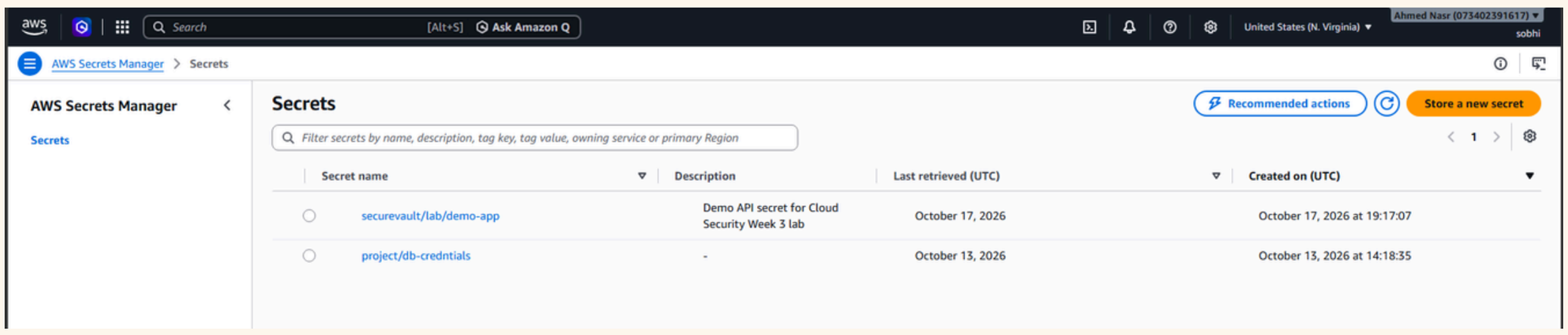


AWS Secret Manager

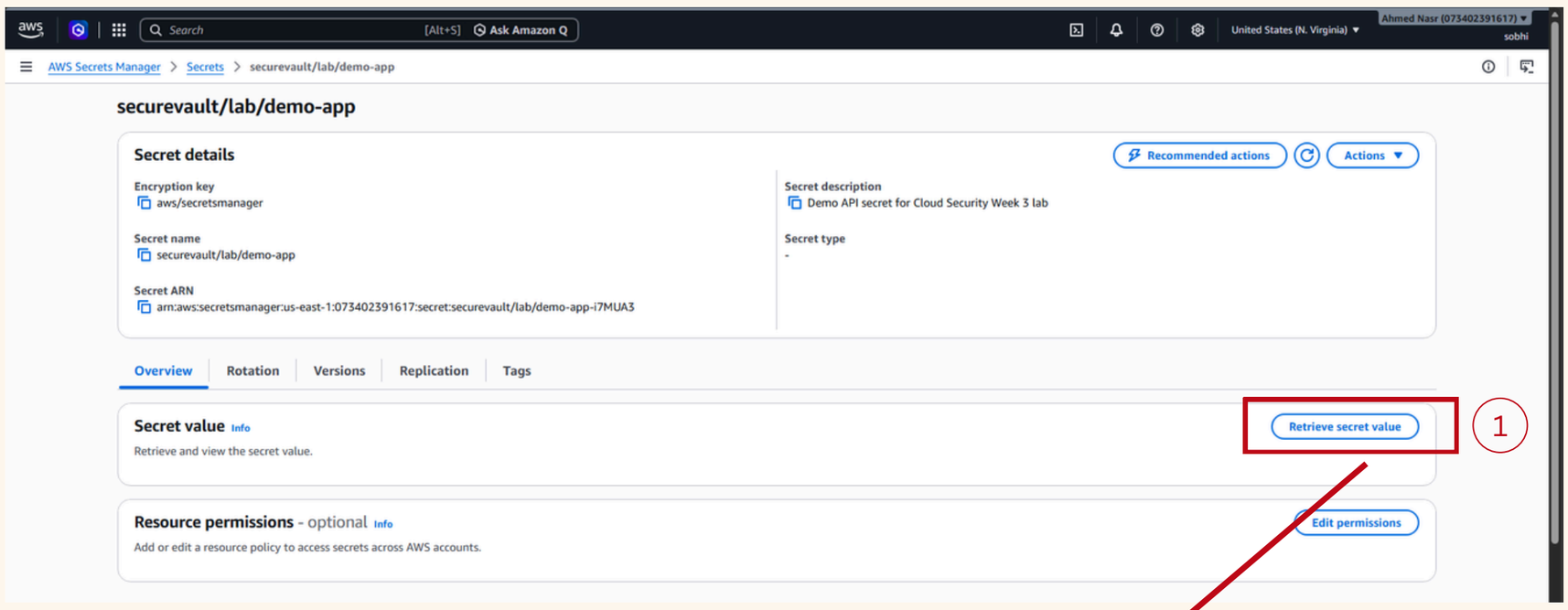
Is the process of securely storing and managing sensitive data such as passwords, API keys, and tokens. using services like *AWS Secrets Manager*, rather than embedding them directly into the code.

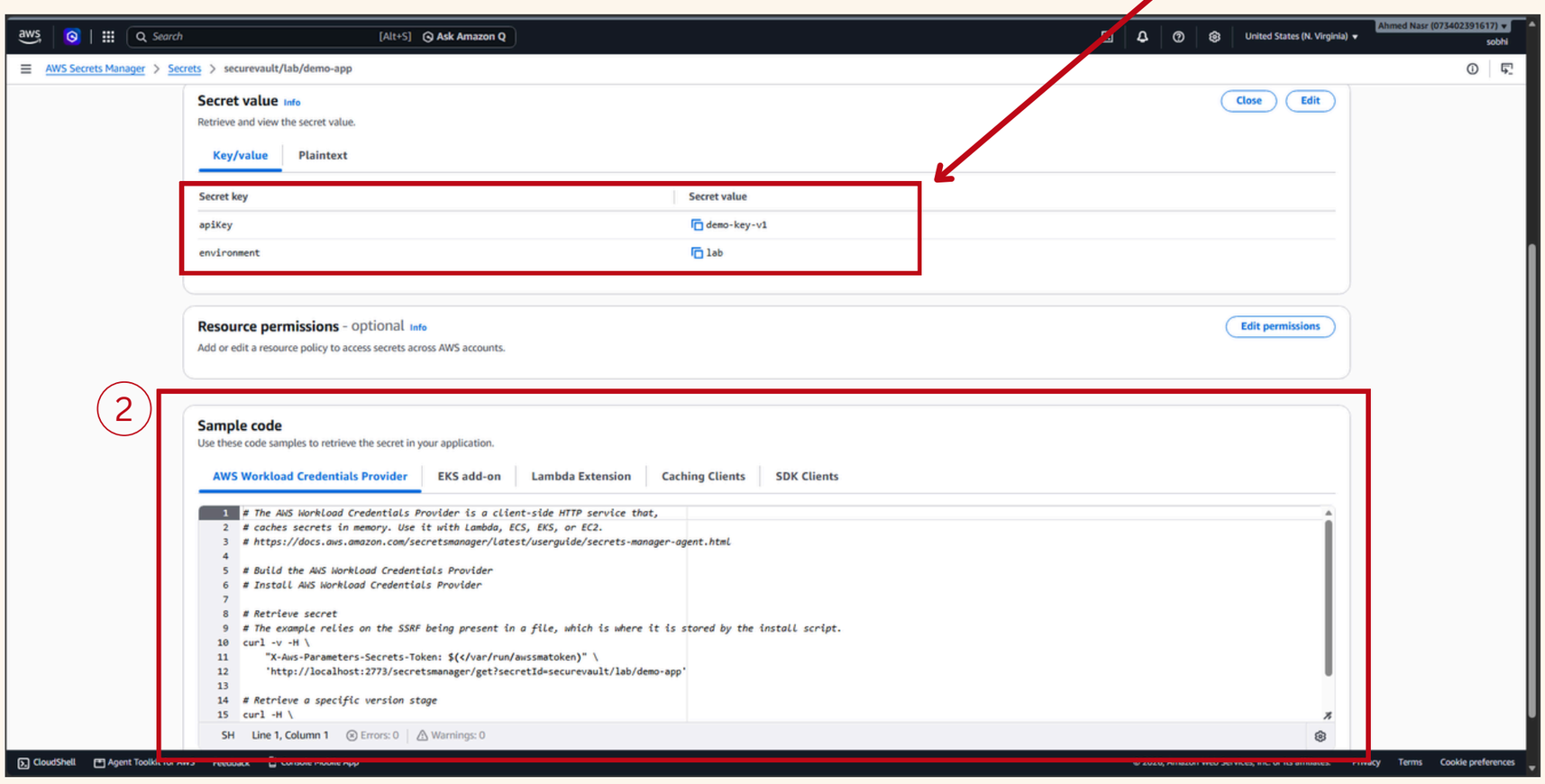
Its importance lies in reducing the risk of secret leakage, controlling access permissions, and periodically rotating these secrets to minimize the impact of a potential compromise, thereby maintaining the security of applications and cloud services.

Create Secret for DB credentials



You can see the secret by click on “Retrieve secret value” or by “Sample Code”





- **Types of Rotation:**
  - Manual Rotation
  - Automatic Rotation

```
MySQL [(none)]>
MySQL [(none)]>
MySQL [(none)]> SELECT VERSION();
+-----------+
| VERSION() |
+-----------+
| 8.4.11    |
+-----------+
1 row in set (0.001 sec)

MySQL [(none)]> CREATE DATABASE rotationlab;
Query OK, 1 row affected (0.004 sec)

MySQL [(none)]> CREATE USER 'rotationapp'@'%' IDENTIFIED BY 'p@ssworD';
Query OK, 0 rows affected (0.024 sec)

MySQL [(none)]> GRANT SELECT, INSERT, UPDATE, DELETE
    -> ON rotationlab.*
    -> TO 'rotationapp'@'%';
Query OK, 0 rows affected (0.013 sec)

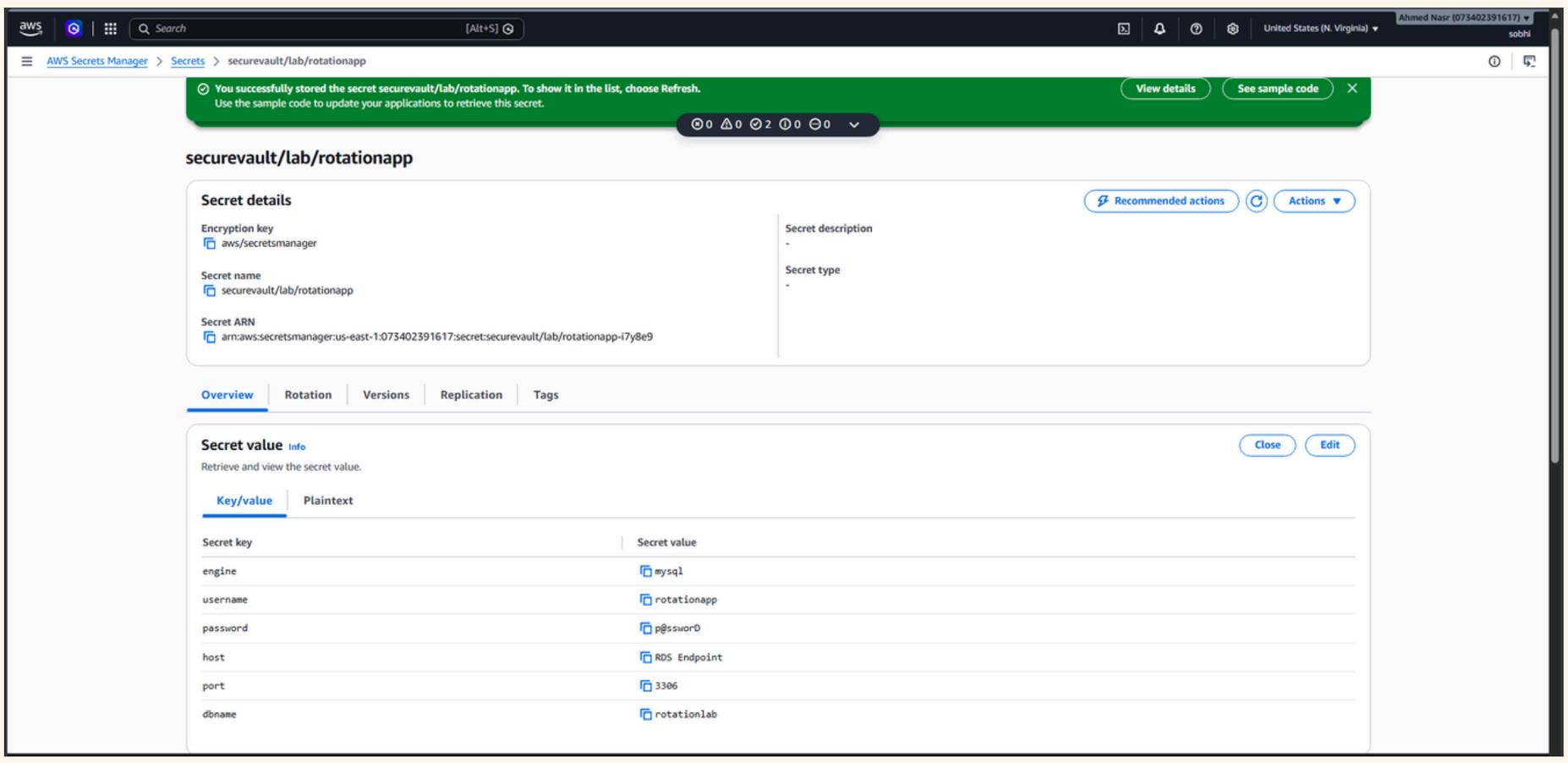
MySQL [(none)]>
MySQL [(none)]> SELECT user, host
    -> FROM mysql.user
    -> WHERE user = 'rotationapp';
+-------------+------+
| user        | host |
+-------------+------+
| rotationapp | %    |
+-------------+------+
1 row in set (0.016 sec)

MySQL [(none)]>
MySQL [(none)]>
```

Databse Name

User Credential

Where are we supposed to maintain this credential ?



What should we do now if the password for this account is leaked?

1. **Manual Rotation**

The person in charge is the one who carries out the change themselves.

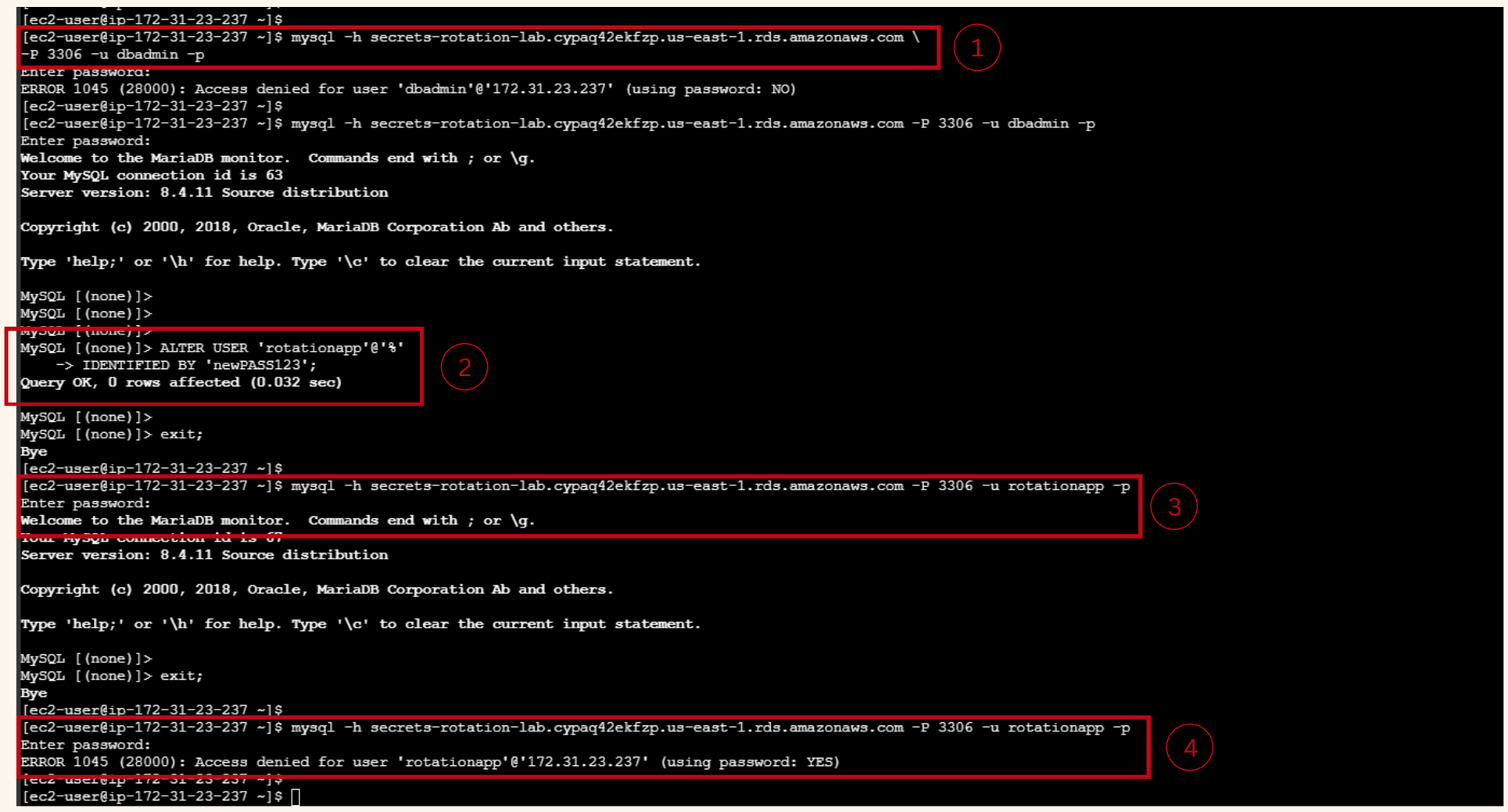
Example:

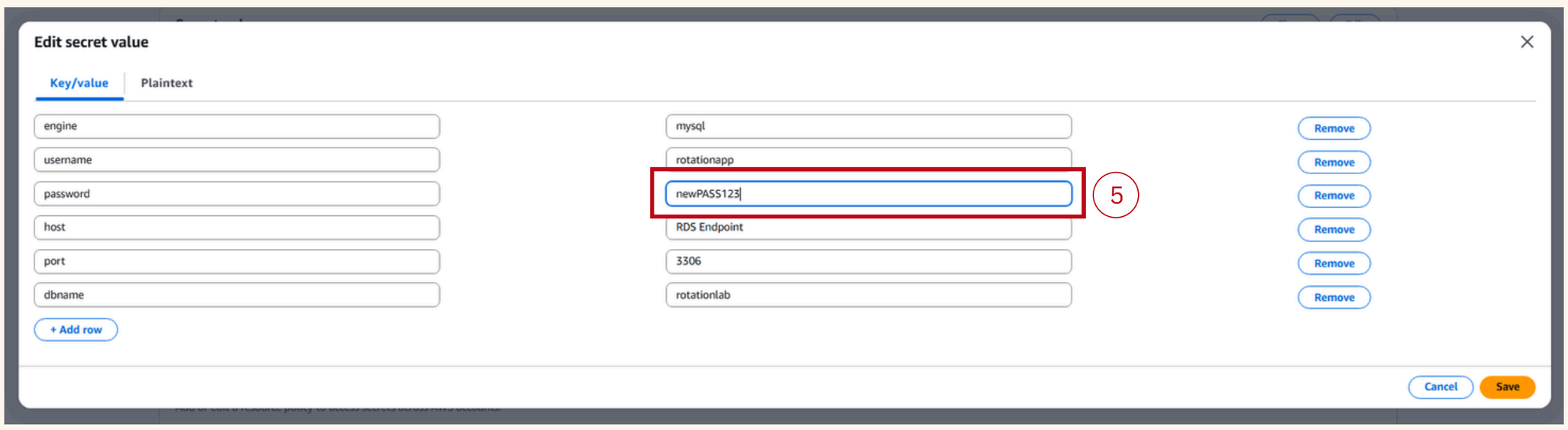
1. Connect to the Database
2. Change account password.
3. Verify that the account is using the new value and is able to connect.
4. Ensure that the old password is no longer valid.
5. Update the value in Secrets Manager.

$ mysql -h secrets-rotation-lab.cypaq42ekfzp.us-east-1.rds.amazonaws.com -P 3306 -u dbadmin -p

$ ALTER USER 'rotationapp'@'%' IDENTIFIED BY 'newPASS123';

Important: If you only change the secret value in Secrets Manager without updating the actual password in the database, the account may fail to connect.



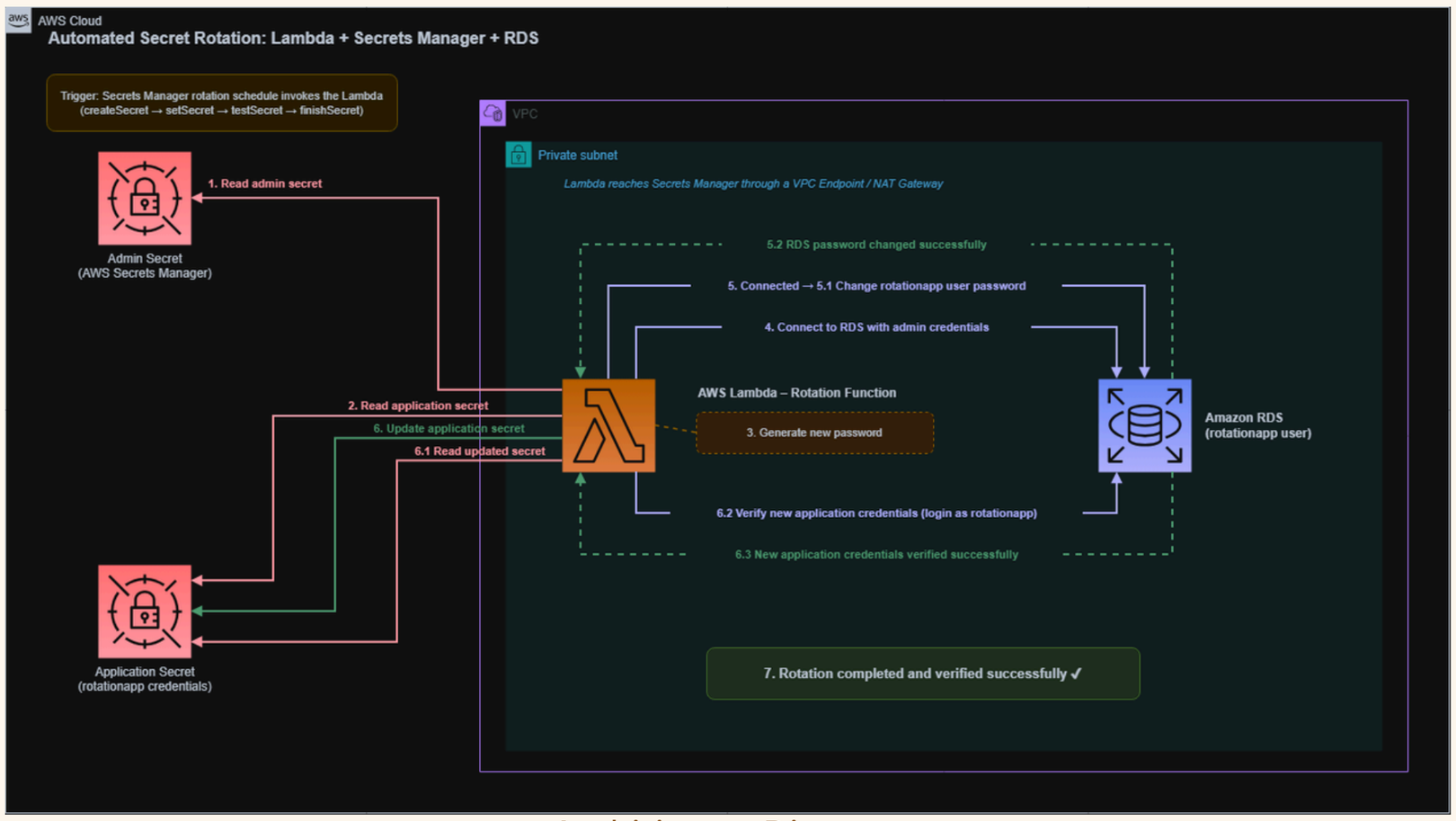


2. **Automatic Rotation**

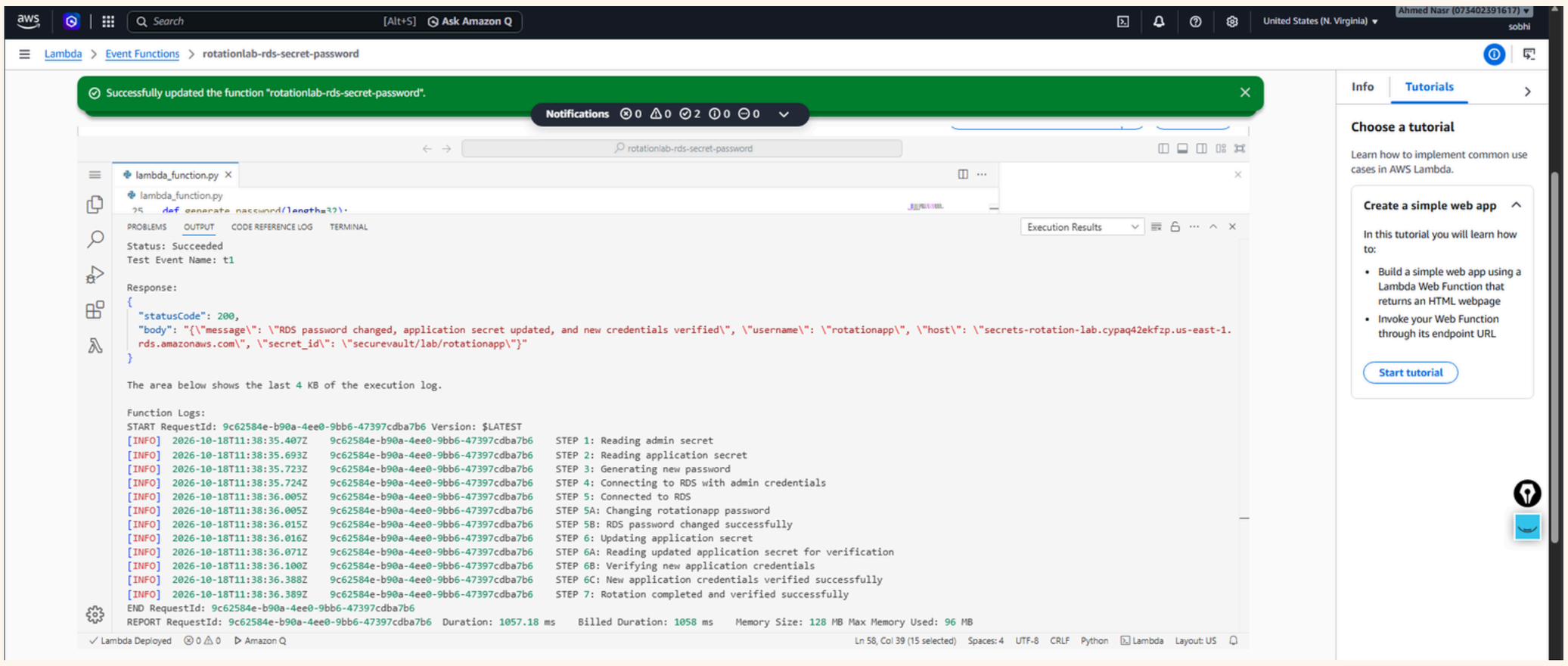
We use Lambda to perform everything we did manually, but in an automated way.

STEPS:

1. Reading admin secret
2. Reading application secret
3. Generating new password
4. Connecting to RDS with admin credentials
5. Connected to RDS
    a. Changing rotationapp user password
    b. RDS password changed successfully
6. Updating application secret
    a. Reading updated application secret for verification
    b. Verifying new application credentials
    c. New application credentials verified successfully
7. Rotation completed and verified successfully



Architicture Diagram



Thanks